RESCUE OPERATIONS SYSTEM (ARDMS)

Advanced Rescue & Disaster Management Suite

PROJECT WORKFLOW & OPERATIONAL MASTER GUIDE

Project Flow and Operational Procedures

Rescue Backup AI Project Workflow & Operational Master Guide

This document defines the **Project Flow, Operational Flow, and Agentic Workflow** for the ARDMS Rescue Backup System.

It ensures that any developer or AI Assistant (e.g., Antigravity) can seamlessly resume work on this project, compile the apps, and deploy the system without losing context.

---

1. Core File & Architecture Locations

- **Web Admin UI**: `system-backend/public/Web ADMIN.html`
- **Rescuer Field App**: `preview-rescuer.html`
- **Public SOS App**: `preview-mobile-app.html`
- **Admin Android App**: `raw_admin.html`
- **Backend Core API/WebSockets**: `system-backend/server.js`
- **Database**: `rescue.db` (SQLite3)

---

2. Agentic Workflow: How Antigravity AI Modifies This Project

If you are an AI assistant (Antigravity) operating on this project on a new laptop, you **MUST** follow these specific workflows when making modifications:

A. Frontend Changes (HTML / CSS / JS)

1. Edit the core web files: `preview-rescuer.html`, `preview-mobile-app.html`, `raw_admin.html`, or `system-backend/public/Web ADMIN.html`.

2. Ensure you handle mangled characters (`ï¸ `) by replacing them with plain text or Lucide icons.

3. Use Explicit filters for JS tasks: e.g., `t.status !== 'completed' && t.status !== 'reassigned'`.

4. Apply CSS `flex-wrap: wrap;` for buttons to prevent overflow.

5. Apply cache-busting (e.g., `?_t=${Date.now()}`) to any Fetch APIs to prevent WebView caching in the Android wrappers.

B. Compilation Process (CRITICAL)

Whenever `preview-rescuer.html`, `preview-mobile-app.html`, or `raw_admin.html` is modified, you must compile them into Android APKs using the local Python build tools.

1. Sync network IPs: `python sync_apps.py`

2. Build Rescuer App: `python build_rescuer.py`

3. Build Public App: `python build_public_apk.py`

4. Build Admin App (Only if asked): `python build_admin_apk.py`

C. Backend Engine Changes

1. WebSocket Updates: Never use a generic `broadcast()` for `NEW_COMMAND` alerts. Always use targeted `socketManager.send(deviceId, ...)` to avoid phantom sirens.

2. Reassignment Buffer: When resetting a task to `pending`, you must clear or ignore its `sos_assignment_history` to prevent false skipping of valid rescuers.

---

3. Operational Workflow (System in Production)

A. Task Classification

- **Critical Tasks**: SOS Alerts, Medical Emergencies, Pregnancy Assist. These trigger sirens and map focus.
- **Normal Tasks**: Food Distribution, Supplies. These are routine and appear silently.

B. AI Auto-Assignment Engine

- The system evaluates incoming SOS requests and assigns them to the nearest available field rescuer within a 50m radius.
- If a rescuer ignores, declines, or times out, the backend auto-reassigns the task to the next closest active unit.

C. Deployment & Network Config

- The backend runs on Node.js port 3001.
- Field users manually enter the local PC/Server IP address on the mobile apps. **Rebuilding APKs is not required when moving to a new Wi-Fi network.**
- Private Cellular/NIB (Network-In-A-Box) setups use Tailscale, Hotspots, or direct SDR eNodeB setups routing to the backend.

---

4. [Fix] Version Control (Git Push Protocol)

Whenever an AI Agent or Developer finishes a task:

1. Validate the compilation (Python scripts).

2. Commit to Git:

```
git add .
git commit -m "[Descriptive Commit Message of the changes made]"
git push origin main
```

3. Maintain documentation integrity.

> **Note for Future Setup**: If restoring this project on a new PC, ensure Python, Node.js (v18+), and JDK 17 are installed. The `.agents/AGENTS.md` file will automatically instruct the Antigravity system on rules and parameters.